

BUSINESS REQUIREMENTS DOCUMENT

Campus Services Portal

A Web-Based Portal for University Students and Administrators

(Student Edition — Expanded & Annotated · Angular Frontend Revision)

Suggested Tech Stack: Angular 19 · ASP.NET Core Web API · SQL Server · Entity Framework Core · JWT **Document Type:** Business Requirements Document (BRD) · **Revision 2.0**

What changed in this revision. The frontend is now **Angular 19** (standalone components, Angular Router, `HttpClient`, Angular Forms) instead of HTML/CSS/vanilla JavaScript with the Fetch API. Everything else — the Web API, the database, JWT authentication, all eight modules, every business rule, every endpoint and the full schema — is unchanged from version 1.0. Frontend setup rules, coding standards and code examples live in the companion document `00-Angular-Frontend-Standard.md`.

1. Project Overview

The Campus Services Portal is a full-stack web application built for a Sri Lankan university. It gives students one place to manage their day-to-day university interactions — hostel applications, lab bookings, event sign-ups, complaints, certificate requests and fee payments — while giving administrators the tools to review, approve and track all of that activity from a single dashboard.

As a student project, this brief is intentionally close to a real client engagement: the requirements are grouped into modules, each with its own data, rules and workflow, and you are expected to design the database, build the API, and connect it to an Angular single-page application — mirroring how a junior full-stack developer would receive work in industry.

1.1 Why This Project Is a Good Fit for This Course

- It exercises the full stack (database → API → Angular UI) end to end.
- Every module reuses the same pattern — one service, one list component, one card component, one form component — so Angular skills compound quickly across modules.
- The portal is a natural home for every concept in Modules 02–07: parent–child lists, status colouring, a shared notification count, role-separated areas, search screens and forms of every complexity.
- Business rules (no double-booking, no duplicate registration) give real practice writing validation in two places — Angular validators for feedback, API checks for enforcement — and understanding why both are needed.
- The scope is large enough to divide across a team or a semester, but each module can also stand alone as a smaller milestone.

2. Objectives

Objective	What This Demonstrates
Develop a REST API	Ability to design resource-based URLs and implement GET/POST/PUT/DELETE actions in ASP.NET Core Web API.
Build a component-based Angular frontend	Ability to split a UI into standalone components with clear <code>@Input()</code> / <code>@Output()</code> contracts, scoped styles and correct lifecycle-hook use.
Bind data between class and template	Ability to use interpolation, property, event and two-way binding, template reference variables and <code>@if</code> / <code>@for</code> control flow.
Transform the UI declaratively	Ability to apply <code>ngClass</code> / <code>ngStyle</code> and write custom directives and pipes instead of formatting data by hand.
Centralise logic in services	Ability to move data access and shared state into injectable singletons consumed through dependency injection.
Navigate a single-page application	Ability to configure routes, parameters, child routes, guards and lazy loading.
Capture and validate user input	Ability to build template-driven and reactive forms with built-in, custom and cross-field validators.
Consume a REST API from Angular	Ability to call the API through <code>HttpClient</code> with typed interfaces and proper error handling.
Perform CRUD operations	Ability to connect an API to a real SQL Server database using Entity Framework Core.
Apply business rules and validations	Ability to translate written rules into server-side and client-side validation logic.

3. User Roles & Permissions

Two roles interact with the system. Keep the permission boundaries below in mind when designing your authentication/authorization logic (role-based checks using JWT claims on the server, route guards in Angular).

3.1 Student

- View and update own profile.
- Apply for hostel accommodation, labs, events, complaints and certificates.
- Track the status of every request they have submitted.
- View own fee history and download receipts.
- Receive notifications relevant to their own activity only.

3.2 Administrator

- Manage (create, update, deactivate) student records.
- Approve or reject hostel, certificate and complaint requests.
- Manage master data: hostels, rooms, labs, event listings, complaint categories.
- View system-wide dashboard statistics across all modules.

- Change the status of any request — this is the only role permitted to do so (see Business Rules).

4. Functional Modules

Each module below follows the same structure: a description, the features to implement, the business rules that govern it, a starter list of REST endpoints, and the database tables it relies on. Treat the endpoint tables as a minimum — you may add endpoints (e.g. pagination, filtering) as needed. Each module closes with a one-line **Angular** note naming the frontend techniques it should demonstrate.

Module 1: Student Profile

Handles student registration and self-service profile management. This module is usually built first because every other module depends on a valid Student record.

Key Features - Register a new student (name, index number, email, faculty, contact details). - Update own profile information. - Search/filter students by name, index number or faculty (admin only). - View a single student's full profile, including a summary of their activity across other modules.

Business Rules - Index numbers and emails must be unique across all students. - Students cannot edit their own index number once registered.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/students/register	Register a new student account.
GET	/api/students/{id}	Get a single student's profile.
PUT	/api/students/{id}	Update a student's own profile.
GET	/api/students?search=&faculty=	Search/filter students (admin only).
DELETE	/api/students/{id}	Deactivate a student account (admin only).

Related Database Table(s): Students

Angular: Reactive registration form with a custom index-number validator and a cross-field password match; a `FormArray` for contact numbers; template-driven login; admin search driven by query parameters; the activity summary is a child component that reloads through `ngOnChanges` when the selected student changes.

Module 2: Hostel Accommodation

Lets students apply for hostel accommodation and lets administrators review applications and assign rooms once approved.

Key Features - Submit a hostel application (preferred hostel, term/semester, special requirements). - View own application status (Pending / Approved / Rejected / Room Assigned). - Administrator reviews and approves or rejects applications. - Administrator assigns a specific room only after approval.

Business Rules - Hostel applications must be approved by an administrator before a room can be assigned. - A room cannot be assigned to more than one active student at a time (check room capacity). - Only administrators can change an application's status.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/hostel-applications	Submit a new hostel application.
GET	/api/hostel-applications/student/{studentId}	View a student's own application(s).
GET	/api/hostel-applications	List all applications, filterable by status (admin only).
PUT	/api/hostel-applications/{id}/status	Approve or reject an application (admin only).
PUT	/api/hostel-applications/{id}/assign-room	Assign a room after approval (admin only).

Related Database Table(s): HostelApplications, Hostels, Rooms

Angular: The room-assignment control sits inside an `@if (status === 'Approved')` branch, with an `@else` explaining why it is unavailable; `ngClass` colours the four statuses; a cross-field validator checks that the term end date follows the start date; a confirmation dialog precedes rejection.

Module 3: Lab Reservation

Allows students to browse available computer/science labs and reserve a specific time slot, and prevents any two students from booking the same slot.

Key Features - View a list of labs and their available time slots. - Reserve a time slot for a chosen lab. - Cancel an existing reservation. - View own upcoming and past bookings.

Business Rules - Lab time slots cannot be double-booked — the API must reject a booking if the slot is already taken. - A student may only cancel their own reservation, not another student's.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/labs	List all labs.
GET	/api/labs/{id}/slots?date=	List available time slots for a lab on a given date.
POST	/api/lab-bookings	Reserve a time slot (rejects if already booked).
DELETE	/api/lab-bookings/{id}	Cancel a reservation (owner only).
GET	/api/lab-bookings/student/{studentId}	View a student's own bookings.

Related Database Table(s): Labs, LabBookings

Angular: The clearest parent-child chain in the portal — a slot-grid parent renders one slot-button child per slot with `@for ... track`, the child colours itself with `ngClass` (available / booked / selected) and emits the click through `@Output()`. A `409 Conflict` must re-fetch the grid so a just-taken slot immediately shows as booked.

Module 4: Event Registration

Displays upcoming university events and manages student sign-ups, ensuring no student registers twice for the same event.

Key Features - View a list of upcoming/past events with details (date, venue, capacity). - Register for an event. - Cancel an event registration. - View own list of registered events.

Business Rules - One student cannot register twice for the same event. - Registration should be blocked once an event reaches its capacity (recommended extension of the base rule).

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/events	List all events.
POST	/api/event-registrations	Register for an event (rejects duplicate registration).
DELETE	/api/event-registrations/{id}	Cancel a registration (owner only).
GET	/api/event-registrations/student/{studentId}	View a student's registered events.

Related Database Table(s): Events, EventRegistrations

Angular: The event card takes an `alreadyRegistered` input and shows a disabled "Already registered" button instead of "Register", so the rule is visible before the click; `ngStyle` drives a capacity bar; the `appHighlight` custom directive gives hover feedback while scanning the list.

Module 5: Complaint Management

Gives students a way to raise complaints (e.g. maintenance, hostel, academic) and gives administrators a workflow to triage and resolve them.

Key Features - Submit a complaint with a category and description. - Track a complaint's status: Pending → In Progress → Resolved. - Administrator updates status and can add a resolution note. - View own complaint history.

Business Rules - Only administrators can change a complaint's status. - A complaint must belong to a valid, pre-defined category.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/complaint-categories	List available complaint categories.
POST	/api/complaints	Submit a new complaint.
GET	/api/complaints/student/{studentId}	View a student's own complaints.
GET	/api/complaints?status=	List complaints filtered by status (admin only).
PUT	/api/complaints/{id}/status	Update status / add resolution note (admin only).

Related Database Table(s): ComplaintCategories, Complaints

Angular: This module carries the **dynamic forms** topic — the complaint form's extra fields are generated from the selected category (Maintenance asks for a building and room; Academic asks for a course code). The custom `shortText` pipe truncates descriptions in the list; a `canChangeStatus` input hides the status control from students.

Module 6: Certificate Requests

Allows students to request official documents — Bonafide Certificate, Transcript or Completion Letter — and track approval progress.

Key Features - Request a certificate by type (Bonafide / Transcript / Completion Letter), with a reason. - Track approval status (Pending / Approved / Rejected / Ready for Collection). - Administrator reviews and approves or rejects requests.

Business Rules - Only administrators can approve, reject, or mark a certificate as ready. - A student should not be able to submit a duplicate pending request of the same certificate type (recommended extension).

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/certificate-requests	Submit a new certificate request.
GET	/api/certificate-requests/student/{studentId}	View a student's own requests.
GET	/api/certificate-requests?status=	List all requests, filterable by status (admin only).
PUT	/api/certificate-requests/{id}/status	Approve/reject/update a request (admin only).

Related Database Table(s): CertificateRequests

Angular: This module carries the **FormArray** topic — a student requests several documents in one submission, adding and removing rows of `{ type, reason, copies }` at runtime, with `min / max` on copies and a validator blocking the same type twice.

Module 7: Fee Payment (Simulation)

Simulates a fee payment process — no real payment gateway is required. Students can view outstanding fees, mark them as paid, and download a receipt.

Key Features - View current outstanding fees. - Mark a fee as paid (simulation — no real transaction). - View payment history. - Download/generate a simple receipt (PDF or on-screen printable view is acceptable).

Business Rules - A fee cannot be marked as paid more than once (prevent duplicate payment records for the same fee item).

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/fee-payments/student/{studentId}	View a student's fees and payment history.
POST	/api/fee-payments/{id}/pay	Mark a fee as paid (simulation).
GET	/api/fee-payments/{id}/receipt	Generate/download a receipt.

Related Database Table(s): FeePayments

Angular: The `currency: 'LKR'` pipe replaces every hand-written money formatter; the pay button is bound to `[disabled]="fee.isPaid || paying"` so the duplicate-payment rule is visible and the double-click race is closed; a confirmation dialog precedes payment. The receipt is a component with an `@media print` block — no PDF library needed.

Module 8: Notifications

Centralises system-generated messages so students always know when something relevant to them has changed — hostel approval, event registration confirmation, complaint updates, certificate approval, and fee reminders.

Key Features - Automatically create a notification whenever a relevant status changes in another module. - List a student's notifications, newest first. - Mark a notification as read.

Business Rules - A student should only ever see their own notifications. - Notifications should be triggered from the relevant module's status-change endpoint (e.g. hostel approval), not created manually by the student.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	<code>/api/notifications/student/{studentId}</code>	List a student's notifications.
PUT	<code>/api/notifications/{id}/read</code>	Mark a notification as read.

Related Database Table(s): Notifications

Angular: This module carries the **singleton service** topic. The unread count appears in the navbar, on the dashboard and in the list, so it lives in one `providedIn: 'root'` service that every component injects — the shared-cart pattern from Module 05. The count refreshes on `NavigationEnd`, which also satisfies the navigation-events requirement.

5. Dashboards

5.1 Student Dashboard

- Current hostel application status.
- Upcoming lab bookings.
- Registered events.
- Certificate request statuses.
- Unread notifications.

5.2 Administrator Dashboard

- Total registered students.
- Pending hostel applications.
- Open complaints by category/status.
- Upcoming events and registration counts.
- Pending certificate requests.
- Summary fee collection statistics.

Learning tip: A dashboard is just several read-only GET endpoints (often one 'summary' endpoint that returns aggregated counts) rendered as cards or a small chart on the frontend — you do not need a separate dashboard table in the database.

Angular: One reusable stat-card component, repeated with `@for`, builds both dashboards; the role decides which one renders. Build the dashboards last — they only aggregate data the other modules already load.

6. Consolidated Business Rules

These rules must be enforced primarily in the API layer (never trust client-side validation alone). The Angular column is feedback only.

#	Rule	Where to Enforce It	Angular support
1	One student cannot register twice for the same event.	<code>POST /api/event-registrations</code> — check for an existing record before inserting.	Disabled "Already registered" button; <code>409</code> refreshes the list.
2	Lab time slots cannot be double-booked.	<code>POST /api/lab-bookings</code> — check slot availability inside a transaction before inserting.	Booked slots render disabled; <code>409</code> re-fetches the slot grid.
3	Hostel applications must be approved by an administrator before room assignment.	<code>PUT /api/hostel-applications/{id}/assign-room</code> — reject if status is not 'Approved'.	Assign-room control sits in an <code>@if</code> branch on Approved.
4	Only administrators can change request statuses.	Role-based authorization (<code>[Authorize(Roles="Admin")]</code>) on every status-changing endpoint.	Admin route guard; status controls hidden from students.
5	A fee cannot be marked as paid more than once.	<code>POST /api/fee-payments/{id}/pay</code> — reject if a payment record already exists.	Pay button disabled once paid; confirmation before submit.

7. REST API & Documentation Requirements

Build full CRUD APIs for Students, Hostel Applications, Labs, Events, Complaints, Certificates, Fee Payments and Notifications, following standard REST conventions:

- Use plural, resource-based URLs (e.g. `/api/students`, not `/api/getStudents`).
- Return proper HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, and **409 Conflict** for double-booking and duplicate-registration attempts.
- Use DTOs to control exactly what data is sent to the frontend (never expose password hashes or internal-only fields).
- Document every endpoint using Swagger/OpenAPI so the frontend can be built against a known contract before wiring up the UI.
- Use JWT-based authentication so the API knows whether a request is from a Student or an Administrator.
- **Enable CORS for the Angular dev server (`http://localhost:4200`) with `AllowAnyHeader()` .** The Angular app is a different origin from the API. This is new in version 2.0 and is the most common first-day blocker.

8. Frontend Requirements

The frontend must be built as a **single-page Angular 19 application** using standalone components, and must talk to the backend exclusively through `HttpClient`. `fetch()`, jQuery and direct DOM manipulation (`document.getElementById`, `innerHTML`) are not permitted. Setup steps, folder structure, coding standards and worked code examples are in `00-Angular-Frontend-Standard.md`.

8.1 Application structure

- A persistent navbar and one `<router-outlet>` in the root component, with a navigation progress bar.
- `core/` holds models, services and guards; `shared/` holds reusable components, directives and pipes; `features/` holds one folder per functional module.
- Shared components used everywhere: navbar, loading spinner, error message, empty state, confirmation dialog, status badge, stat card.
- Every list screen is a parent-child pair: the parent owns the array and the API call, the child renders one record and emits events upward.

8.2 Routing

Element	In this portal
Redirect	<code>' ' → /dashboard</code>
Route parameters	<code>/labs/:id/slots</code> , <code>/complaints/:id</code> , <code>/events/:id</code> , <code>/fees/:id/receipt</code>
Query parameters	<code>/admin/students?search=&faculty=</code> , <code>/admin/complaints?status=</code> , <code>/events?upcoming=</code>
Child & nested routes	Each feature area (labs, events, hostel, complaints, certificates, fees, admin) has a shell with its own outlet
Wildcard	<code>**</code> → 404 page, always the last entry
Guards	<code>authGuard</code> past login; <code>adminGuard</code> on the <code>/admin</code> tree
Lazy loading	<code>loadComponent</code> on every feature route
Navigation events	<code>NavigationStart</code> / <code>NavigationEnd</code> drive the progress bar and refresh the notification count

8.3 Services

One `@Injectable({ providedIn: 'root' })` service per module — `Student`, `Hostel`, `Lab`, `Event`, `Complaint`, `Certificate`, `Fee`, `Notification`, `Dashboard` — plus `AuthService`. Every service injects `HttpClient`, reads the base URL from `environment.apiUrl`, and returns typed `Observable<T>`. No component injects `HttpClient` directly. The notification service also holds the shared unread count.

8.4 Forms

Form	Approach	Key validation
Login; event and fee confirmations; admin master data	Template-driven	<code>required</code> , <code>email</code> , <code>min</code>

Form	Approach	Key validation
Student registration	Reactive	Custom index-number and email-domain validators; cross-field password match
Profile update	Reactive	<code>FormArray</code> of contact numbers
Hostel application	Reactive	Cross-field term end after start
Lab booking	Reactive	Custom future-date validator
Complaint	Reactive, dynamic	Fields generated from the selected category
Certificate request	Reactive	FormArray of documents; <code>min / max</code> copies; no duplicate type

Both approaches must appear in the project. Errors are shown only after a field is touched, and the submit button is disabled while a request is in flight.

8.5 Directives and pipes

Custom directives: `appHighlight` (hover feedback), `appStatusColor` (background from a bound status). Custom pipes: `shortText` (parameterized), `statusLabel`, `timeRange`. Built-in pipes in required use: `titlecase`, `uppercase`, `currency: 'LKR'`, `number`, `date`, `slice`. `ngClass` and `ngStyle` carry all conditional styling — status badges, slot availability, capacity bars, overdue fees.

8.6 User interface rules

- Provide search and filtering wherever a list of records is shown, driven by query parameters so the filtered view is linkable and survives a refresh.
- Every screen that calls the API renders four states: loading, error, empty and data.
- Show a confirmation dialog before every destructive or irreversible action.
- Validate forms on the client for immediate feedback, in addition to server-side validation.
- Handle `409 Conflict` as a normal outcome: show the message and refresh the affected list.
- Use responsive layout techniques so the portal works on both desktop and mobile.
- Label every input, and never signal a status by colour alone.

9. Database Design

The tables below are a suggested starting schema. You are encouraged to refine column names/types and add audit fields (`CreatedAt` , `UpdatedAt`) as you design your own ER diagram.

Table	Purpose	Key Relationships
<code>Students</code>	One row per student account.	Referenced by almost every other table via <code>StudentId</code> (FK).
<code>HostelApplications</code>	One row per hostel application submitted.	FK → <code>Students</code> , FK → <code>Hostels</code> .
<code>Hostels</code>	Master list of hostel buildings.	Referenced by <code>Rooms</code> and <code>HostelApplications</code> .

Table	Purpose	Key Relationships
Rooms	Individual rooms within a hostel, with capacity.	FK → Hostels ; referenced by HostelApplications once assigned.
Labs	Master list of labs available for reservation.	Referenced by LabBookings .
LabBookings	One row per reserved time slot.	FK → Labs , FK → Students .
Events	Master list of university events.	Referenced by EventRegistrations .
EventRegistrations	One row per student registered for an event.	FK → Events , FK → Students .
ComplaintCategories	Master list of complaint categories.	Referenced by Complaints .
Complaints	One row per complaint submitted.	FK → Students , FK → ComplaintCategories .
CertificateRequests	One row per certificate request.	FK → Students .
FeePayments	One row per fee item / payment.	FK → Students .
Notifications	One row per notification generated for a student.	FK → Students .

10. Non-Functional Requirements

Category	Expectation
Security	Passwords hashed (never stored in plain text); role-based authorization on admin-only endpoints. JWT held in memory with a <code>sessionStorage</code> fallback, cleared on logout. Route guards control navigation; <code>[Authorize]</code> controls data.
Performance	List endpoints support basic paging so large tables don't slow down the UI. Feature routes are lazy-loaded; every <code>@for</code> uses <code>track</code> .
Usability	Consistent navigation and clear feedback messages across all modules; loading, error, empty and data states everywhere; confirmation before irreversible actions.
Maintainability	Layered backend architecture (Controllers → Services → Repositories). On the frontend, all HTTP in services, all formatting in pipes, no component over ~200 lines.
Responsiveness	Every screen usable from 360px to desktop width.
Accessibility	Labels on every input, sensible focus order, status never signalled by colour alone.
Portability	The SQL script and README must allow another developer to set up the project from scratch; <code>npm install && ng serve</code> runs the frontend with only <code>environment.ts</code> to edit.

11. Suggested Build Order (For a Solo or Small-Team Project)

- Design the full database schema and relationships before writing any C# code.
- Build the Student Profile module end-to-end on the API side and publish it in Swagger.
- Implement authentication (registration/login, JWT issuing) **and enable CORS** before building any Angular screen.
- Scaffold the Angular workspace, install Tailwind, generate environments, enable `provideHttpClient()`.
- Write the model interfaces from the Swagger schema, then build the shared components every feature needs.
- Build the Student Profile Angular feature end-to-end — it becomes the template every other module copies.
- Build the remaining modules one at a time, always finishing API + Swagger test + Angular feature together before moving on.
- Add Notifications and the shared unread count last of the modules, since it depends on status changes from the others.
- Build the dashboards last — they only aggregate data that already exists.
- Finish with README, SQL script export and a walkthrough test of every business rule, including a deliberate double-booking attempt on the same lab slot from two browsers.

12. Deliverables

Deliverable	Details
Source Code	Full ASP.NET Core Web API solution and Angular 19 workspace, in a Git repository, with <code>node_modules</code> excluded.
SQL Script	A script that creates the full schema and seeds minimal sample data (e.g. hostels, labs, complaint categories).
REST API	All CRUD endpoints described in Section 4, secured and returning correct status codes.
Angular Frontend	Responsive single-page application for every module, wired to the API through <code>HttpClient</code> . <code>ng build</code> completes with no errors.
Frontend documentation	Component inventory (selectors, inputs, outputs), routing table, service inventory and forms register.
Module coverage evidence	The Campus Services Portal column of <code>05-Angular-Module-Coverage-Matrix.md</code> , with a file path against each concept.
Swagger Documentation	Auto-generated OpenAPI docs covering every endpoint, with example requests/responses.
README	Setup instructions, tech stack, ER diagram, CORS and environment configuration, and a short description of each module.